

The Archive

A Living Spatial Mindspace

Technical Report

Author	Akarsh Garg
Source Code	https://github.com/agcode-maker/living-spatial-mindspace.git
Video	https://youtu.be/U8hX6osK43A?si=A0H3x2SucztzbFu5

Submitted for: Living Spatial Mindspace Project Statement
July 2026

Table of Contents

Table of Contents.....	2
1. Introduction & Problem Understanding	3
2. Objectives	3
3. Technology Stack & Justification.....	3
4. System Architecture	3
4.1 Component Responsibilities	4
4.2 Data Model	4
5. Feature Implementation Against the Brief.....	4
6. Agentic AI System Design	4
7. Key Design Decisions & Trade-offs.....	5
7.1 Crosshair + keyboard interaction, not mouse-click	5
7.2 Two-tier persistence strategy	5
7.3 Independent constellation-view camera.....	5
7.4 On-demand rather than continuous AI reasoning	5
8. Challenges Faced & Solutions.....	6
9. Testing & Quality Assurance.....	6
10. Screenshots	6
11. Future Improvements.....	8
12. Conclusion.....	8

1. Introduction & Problem Understanding

The Living Spatial Mindspace brief asks for a small interactive 3D application that reimagines how a person interacts with digital information - moving away from flat, screen-bound interfaces toward a persistent, spatial personal workspace. The brief is explicit that the result should feel "useful, intentional, and alive," not merely a visually impressive tech demo, and mandates an embedded agentic AI system that participates meaningfully in the user's workflow rather than existing as a bolted-on chat panel.

The Archive is built around a single unifying idea in response to that brief: knowledge objects placed in the space form an actual 3D knowledge graph, and an AI curator lives inside that same graph as a physical entity, actively helping shape it. Every required feature - navigation, object manipulation, persistence, atmosphere, and the agentic AI - was designed to reinforce that one idea rather than exist as separate, disconnected checklist items.

2. Objectives

- Build a persistent, explorable 3D personal workspace usable on modest, integrated-graphics hardware.
- Represent notes, tasks, ideas, and images as physically manipulable spatial objects, with no conventional menu-driven interaction.
- Implement a knowledge-graph system where relationships between objects are visible, spatial, and user-constructed.
- Embed a genuinely agentic AI curator that reasons about the workspace, remembers across sessions, and participates as a spatial entity.
- Make the environment visibly reactive - both to individual object usage and to the overall state of the space.
- Package the result as a real, distributable Windows executable.

3. Technology Stack & Justification

The dominant constraint on every technical decision was hardware: development took place on an 11th-generation Intel Core i5 laptop with 8GB RAM and Intel Iris Xe integrated graphics - no dedicated GPU. This ruled out Unity and Unreal, both of which impose meaningful GPU and build-time overhead unsuited to that machine, in favor of a browser-rendered WebGL stack that runs comfortably on integrated graphics and iterates in seconds rather than minutes.

Layer	Choice	Reason
Rendering	React Three Fiber (Three.js)	Runs in-browser on integrated graphics; no engine install or long compiles
State	Zustand	Single lightweight store shared by rendering, persistence, and the AI agent
Desktop packaging	Electron + electron-builder	Produces a real Windows executable from the same web codebase
AI	Google Gemini API (gemini-3-flash-preview)	Fast, generous free tier, sufficient for grounded short-form reasoning
Persistence	localStorage / JSON file	Same abstraction works in-browser and in the packaged desktop app

4. System Architecture

The application has no backend server - all logic, including AI orchestration, runs client-side. A single Zustand store is the entire application's source of truth; every subsystem below reads and writes through it rather than maintaining separate local state.

4.1 Component Responsibilities

- 3D World (React Three Fiber): renders the scene, camera, lighting, and all knowledge objects; owns crosshair-based targeting and carry/manipulate interactions.
- Zustand Store: the single source of truth - object list, curator memory log, session/view state, and every mutation path (discrete vs. continuous, see Section 6).
- Persistence Layer: an abstraction that transparently backs the store with localStorage in the browser or a real JSON file when running inside Electron.
- AI Curator Agent: an isolated module that builds prompts from live workspace state and calls the Gemini API; the only place any LLM-provider-specific code exists.
- Electron Shell: the desktop wrapper (main process + preload script) that turns the same web app into a distributable executable with file-backed persistence.

4.2 Data Model

The entire world is one array of knowledge objects. There is no separate graph data structure - the graph is simply the links array each object carries, which keeps persistence trivial (a single JSON blob) and guarantees the renderer, the persistence layer, and the AI curator are always reasoning about exactly the same data.

Object fields: id, type (note / task / idea / image), label, position [x, y, z], scale, rotationY, color, links [otherId, ...], createdAt, lastTouched.

5. Feature Implementation Against the Brief

The table below maps each requirement from the problem statement to its concrete implementation.

Requirement	Status	Where implemented
Navigate & explore freely, smooth camera	Done	FlightControls.jsx - WASD + pointer-lock mouse look
Multiple interactable zones	Done	Whole graph is the interactable space; constellation view surfaces structure
Create / place / move / organize objects spatially	Done	Interactions.jsx - spawn, carry, link, rename, delete
Rotate / scale objects	Done	Scroll to scale, Z/X to rotate while carrying
No conventional menus for interaction	Done	Crosshair + keyboard driven throughout
Persistent world state	Done	persistence.js - localStorage (browser) / JSON file (Electron)
Meaningful spatial system	Done	Object-based knowledge graph + constellation view
Agentic AI, embedded not overlay	Done	CuratorEntity.jsx - physical entity, context-driven movement
Atmosphere & environmental feedback	Done	Living-memory glow + organization-reactive lighting/fog
Main menu: enter / save / reset / continue	Done	MainMenu.jsx

6. Agentic AI System Design

The curator is deliberately not a chat sidebar. It is rendered as a physical light entity inside the 3D scene, and its movement is driven by a priority chain read from live application state each frame, rather than an independent random wander - this is what makes its behavior legible as "doing something" rather than merely decorative:

- While awaiting an API response: holds still ("thinking").
- While a cluster suggestion is pending: hovers directly between the two objects it is proposing to connect.
- While the user is carrying an object: stays near that object.
- Otherwise: drifts toward whatever the user is currently looking at.
- With no other signal: idly drifts among the user's existing objects.

Every provider-specific API detail is isolated inside a single function in `src/ai/curatorAgent.js`, so the rest of the application - and the prompts themselves - are provider-agnostic. Three functions cover the required behaviors:

- `summarizeSessionStart(objects, log)` - contextual memory across sessions.
- `suggestClusters(objects)` - proactive, semantically-grounded organization suggestions, returned as a preview the user explicitly accepts or dismisses rather than silently auto-applied.
- `askCurator(message, objects, log)` - free-form natural-language interaction grounded in real workspace state.

Behavior	How The Archive implements it
Contextual memory across sessions	<code>curatorLog</code> persisted to disk; session-start greeting references it directly
Autonomous / proactive assistance	'G' triggers unprompted-feeling analysis of the whole space, not a per-object action
Semantic understanding of content	Clustering prompts are grounded in real object labels, not random pairing
Environmental adaptation	Atmosphere (lighting/fog) reacts to graph organization; objects glow/dim by use
Natural-language interaction	'T' opens a grounded chat - answers reference actual workspace state

7. Key Design Decisions & Trade-offs

7.1 Crosshair + keyboard interaction, not mouse-click

The camera uses pointer-lock controls for FPS-style look-around, which captures the mouse for camera rotation and leaves no free on-screen cursor to click precise 3D targets with. Rather than fight that, the app raycasts once per frame from the exact center of the screen to determine what is being looked at, and all manipulation happens via keyboard shortcuts. This keeps every interaction genuinely spatial - you act on what you are looking at - and satisfies the brief's requirement that interaction happen "naturally inside the 3D world rather than through standard UI panels."

7.2 Two-tier persistence strategy

Discrete, user-intentional edits (rename, link, delete) persist to disk immediately. Continuous edits - moving, scaling, or rotating a carried object - update the in-memory store every animation frame but only persist once, when the object is dropped. Persisting on every frame would mean up to 60 disk writes per second in the Electron build, which is both wasteful and a potential source of stutter.

7.3 Independent constellation-view camera

The bird's-eye "constellation view" (toggled with C) uses a second, independent camera with free-orbit controls rather than repurposing the first-person camera. This avoided any conflict with the pointer-lock controls' internal rotation state, at the cost of slightly more setup code - a deliberate robustness-over-cleverness trade-off given the project timeline.

7.4 On-demand rather than continuous AI reasoning

The curator's clustering analysis runs only when the user explicitly requests it, rather than continuously in the background. This keeps API usage predictable and avoids the jarring experience of objects re-linking themselves unprompted while the user is doing something else.

8. Challenges Faced & Solutions

Challenge: Click-based selection was unreliable once pointer-lock camera controls were introduced, since a locked pointer has no free on-screen cursor.

Solution: Replaced mouse-click selection with continuous crosshair raycasting plus keyboard actions - a standard FPS interaction pattern that also better matches the brief's emphasis on non-menu-driven interaction.

Challenge: Link lines visually detached from an object while it was being carried, despite the underlying position data updating correctly every frame.

Solution: The root cause was a raw Three.js BufferGeometry not being flagged for a GPU buffer update on every React re-render. Switching to a declarative Line component designed for dynamic point updates resolved it.

Challenge: Scroll-to-scale for carried objects also triggered the browser's native pinch/ctrl-scroll zoom.

Solution: The wheel event listener was registered as passive, which silently prevents `preventDefault()` from taking effect; switching it to non-passive and explicitly calling `preventDefault()` while an object is being carried resolved it.

Challenge: Packaging with electron-builder failed on Windows with a "cannot create symbolic link" error while extracting a bundled code-signing helper package.

Solution: This is a known Windows restriction on symlink creation without elevated privileges. Enabling Developer Mode (or running the build from an Administrator terminal) grants the necessary privilege; the project does not require code signing at all for this submission.

9. Testing & Quality Assurance

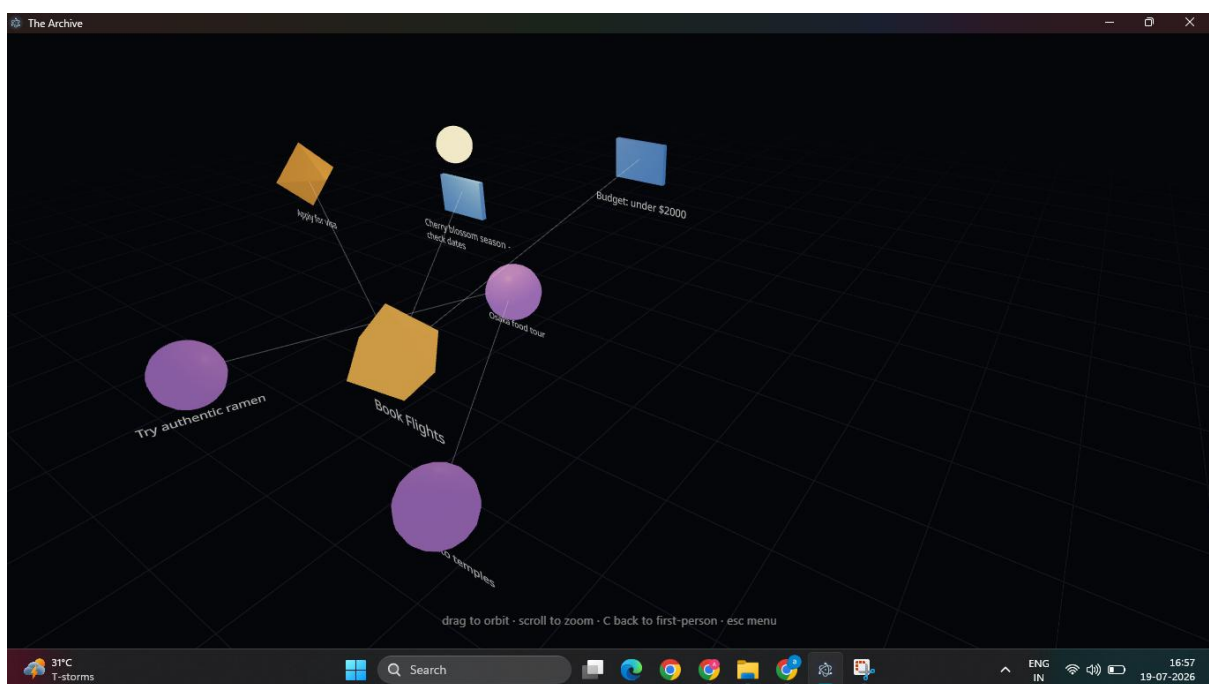
Testing was manual and scenario-driven, prioritized by likelihood of occurring during grading rather than exhaustive coverage, given the project timeline. Verified scenarios include:

- A full session cycle: onboarding, object creation, linking, curator interaction (both organize and chat), constellation view, return to menu, and re-entry with state intact.
- The packaged executable specifically (not the dev server), including a fresh, file-backed persistence path distinct from the browser's `localStorage`.
- Edge cases around interrupted actions - deleting an object mid-link, deleting an object referenced in a pending AI suggestion, canceling the rename panel.
- Rapid input spamming (repeated pick-up/drop, repeated AI requests) to confirm the busy-state guard prevents overlapping API calls.

10. Screenshots



Knowledge objects placed and linked in the space, forming the visible knowledge graph.



Constellation view - a bird's-eye perspective of the entire knowledge graph at once.

11. Future Improvements

- Richer per-object visuals - e.g. notes rendering their actual text content on their physical face.
- Continuous, low-frequency background clustering analysis in addition to the on-demand pass.
- Multi-agent variants of the curator (e.g. a separate "archivist" that only manages images/files).
- Cross-device sync of the save file, beyond the current single-machine local persistence.

12. Conclusion

The Archive delivers every mandatory requirement in the Living Spatial Mindspace brief - free spatial navigation, menu-free object manipulation including rotation and scaling, persistent world state, a meaningful spatial knowledge-graph system, an embedded agentic AI with multiple demonstrated behaviors, and a reactive atmosphere - while remaining runnable on genuinely modest hardware. The central design choice, treating the AI curator as a physical inhabitant of the same knowledge graph the user builds rather than a separate feature, was intended to make the required systems reinforce rather than merely coexist with each other.